iacc instituto profesional

EVALUACIÓN

Evaluación de Sistemas QA
Semana 2

Amparo Valdivia
22, enero de 2026
Técnico en Análisis y Programación.

DESARROLLO:

En contexto la automatización de Pruebas para una Aplicación web, perteneciente a una empresa de desarrollo de software, que realiza el cálculo de Fuerza.

El cálculo es por medio de la fórmula de la Ley de Newton:

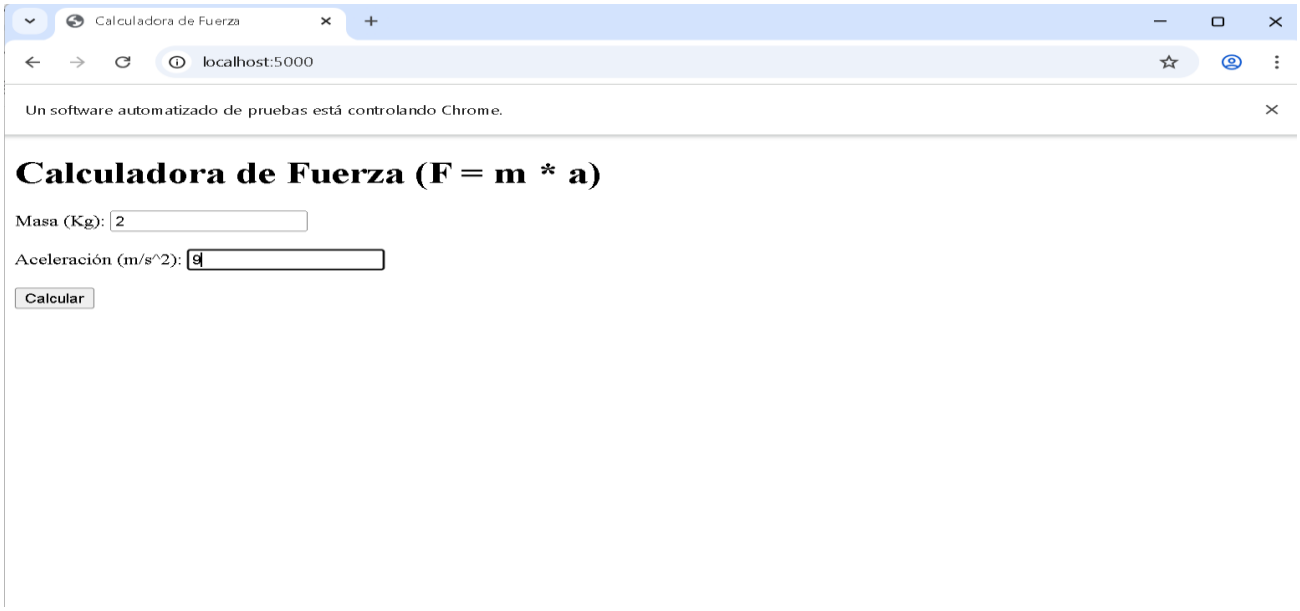
Fuerza = masa x aceleración.

Instrucciones para el equipo responsable de las pruebas, el administrador Amparo Valdivia:

- A modo introductorio para diseñar pruebas automatizadas para garantizar el correcto funcionamiento de esta aplicación.

  Se realizaron las primeras pruebas con Selenium webdriver, para las pruebas funcionales, los errores encontrados, fueron documentados en Mantis, asignados a desarrollo para su revisión y resolución, una vez que estuvieron listos, se entregaron al equipo de producción para la integración y la realización de pruebas de regresión automatizadas.

  Imagen de ejemplo de una de las pruebas realizadas.



Posteriormente se realizaron pruebas de sistema con Selenium IDE, que se detallan más adelante.

1. ¿Cuáles son los componentes principales que debemos considerar al examinar los frameworks de automatización de pruebas, especialmente en el contexto de la aplicación de cálculo de fuerza?

Durante el proceso, una vez que el código fue estructurado en módulos dentro de un entorno de desarrollo y realice las primeras pruebas manuales en consola para clarificar si existía algún error que se pudiera solucionar a simple vista y en una primera revisión inicial, surgió la necesidad de poder observar como se comportaba el sistema y la función de cálculo mediante una prueba automatizada que pudiera hacer correr el algoritmo, en este sentido, solo buscaba realizar una prueba funcional y comprobar si la función cumplía su cometido, “El cálculo”, sin embargo a medida que la prueba automatizada iba procesando el flujo, note claramente errores que viendo el código o el sistema en consola no podía ver, es importante estar del lado del usuario, también valoré profundamente la rapidez con que podía reparar el error, implementar la solución y muy importante probar otra vez, en este sentido las pruebas automatizadas son un gran aporte al programador y a todo el equipo de trabajo, entonces comprendí que la automatización al ser una prueba de rápida implementación y rápida respuesta entrega gran cantidad de datos, esta fue la fundamentación perfecta para asumir que la documentación en entornos como Mantis o Jira son precisos, estos permitieron llevar un orden en el proceso de las pruebas y una orientación al logro.

Sin embargo en momentos en que ya se habían realizado varias pruebas automatizadas con Selenium webdriver y los reportes en Mantis con notas, imágenes e incluso código adjunto, se hizo imprescindible automatizar el proceso aún más, evidenciar a través de un video o grabación el proceso, me permitía analizar mas a fondo las necesidades a cumplir, la forma en que se procesan los flujos, mejorar el sistema y también la línea por línea, sin

tener que hacer un breakpoint manual, que si fue necesario realizar en Visual Studio Code, dicho esto, podía repetir el proceso exactamente igual como ocurrió, que no es lo mismo que lo realice alguna persona que hubiera leído como repetir el proceso en la documentación, porque pudiera haber algún salto o error en el proceso, no pudiéndose repetir el mismo error, entonces las pruebas automatizadas mediante frameworks personalmente considero que enriquecen el punto de vista de quien procesa y observa la prueba, también encontré en esta experiencia que cada punto queda muy bien registrado, sin embargo quisiera recalcar que para proporcionar una prueba excepcional es necesario internarse en el aprendizaje del framework para saber que elemento es necesario para la prueba a realizar.

- Motor de Ejecución (WebDriver)
- Gestión de Objetos (Locators), poder identificar elementos por su id, como, por ejemplo, como id=masa e id=error.
- La integración con herramientas como MantisBT, donde la automatización alimenta la documentación técnica y la documentación técnica organiza a el equipo para llegar a realizar nuevas pruebas automatizadas.
- Como se van a realizar las validaciones, éxito o fallo, como el assert element present.
- Alta capacidad de repetitividad.

3. ¿Cómo podemos aplicar los principios de desarrollo de scripts y pruebas automatizadas para mejorar la calidad del sistema de cálculo de fuerza, teniendo en cuenta los requisitos específicos y los escenarios de prueba identificados en nuestro caso práctico?

Para realizar una síntesis del análisis de las pruebas que puedan realizarse a un programa, mediante el análisis, se ha presentado en esta actividad la necesidad de gestionar el flujo de trabajo, ya que he de saber que a medida que es necesario notificar un error y cuando

es dado por finalizado, el conocimiento durante este proceso puede surgir como ha sido el caso, el hallazgo de nuevos errores los que por correcta estructura, debe ser notificado como un nuevo asunto, pero pudiese darse el caso que el equipo de trabajo decida continuar adicionando los asuntos para que estos se mantengan dentro del mismo flujo, en ambos casos mantener al equipo informado de cada paso, es parte del proceso de calidad también, el proceso, Más ejemplos de mejoras en el sentido del flujo, palabras de capítulo 12 y Organización para mejorar la calidad y la productividad capitulo 16 de W. Edwards Deming Calidad, Productividad y Competitividad, La salida de la crisis, he de pensar que en la separación de responsabilidades puede estar la más rápida acción y que el desarrollo de una solución a un error cumpla con la norma de independencia para no afectar el flujo del sistema, es por este motivo que para cada nuevo error he decidido crear un nuevo issue.

En la aplicación de principios entonces está:

- Trazabilidad, y la gestión de flujo, La independencia de los fallos. Al decidir crear un nuevo issue para el error de validación de datos en lugar de mezclarlo con el de usabilidad, se garantizó que cada solución cumpla con la norma de independencia.
- Robustez Técnica, configurar el entorno de trabajo, la automatización requiere conocer el entorno del software.
- Validación de Escenarios, se validan los resultados con enteros, números negativos y escenarios negativos masa 0, robustez, al ingresar strings y probar el sistema.

Durante el primer error encontrado de usabilidad, el ciclo de vida del error fue documentado con Mantis, y a cada nuevo registro se le asignó un número de reporte como nota adjunta, el segundo error es de validación de datos,

En pruebas con scripts como Selenium webdriber se pueden comprobar situaciones como si intenta entrar al puerto 5000 pero Flask está en el puerto 8000, la prueba va a fallar, si por alguna razón el puerto 5000 está ocupado por otro programa, y las puebas de Selenium como resultado sean exitosas, la URL en el script de prueba debe coincidir exactamente con el puerto que Flask está usando. Flask: http://127.0.0.1:5000 por el código proporcionado puerto por default.

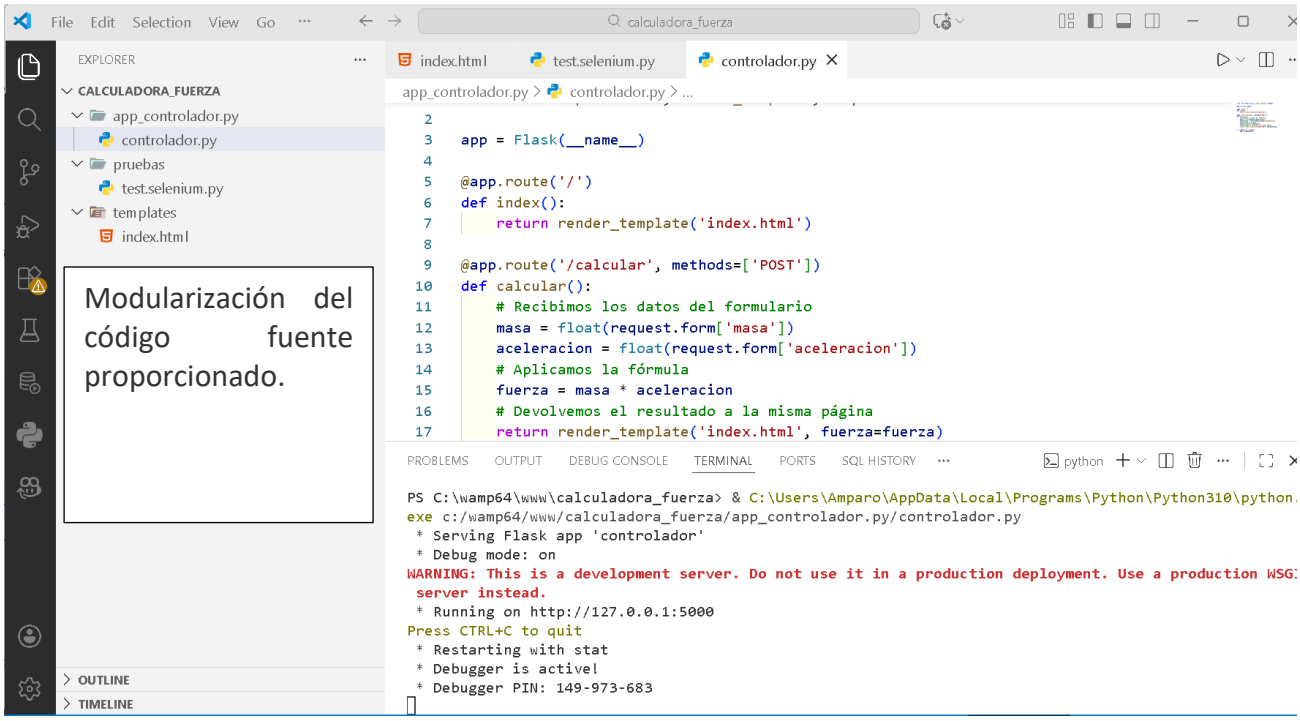
En Selenium: driver.get("http://localhost:5000/")

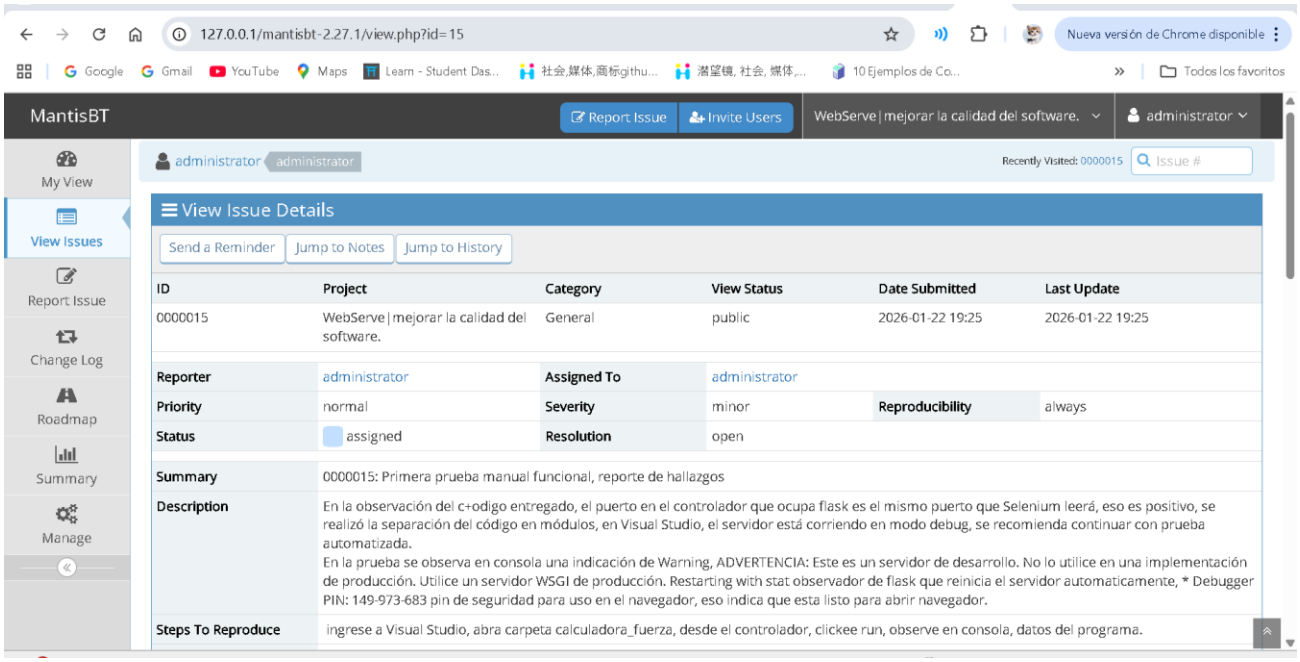
- Para probar si algún programa está en el puerto, escribir en consola netstat -ano | findstr :5000,
- Para abrir otro puerto if __name__ == '__main__':

app.run(debug=True, port=8080)

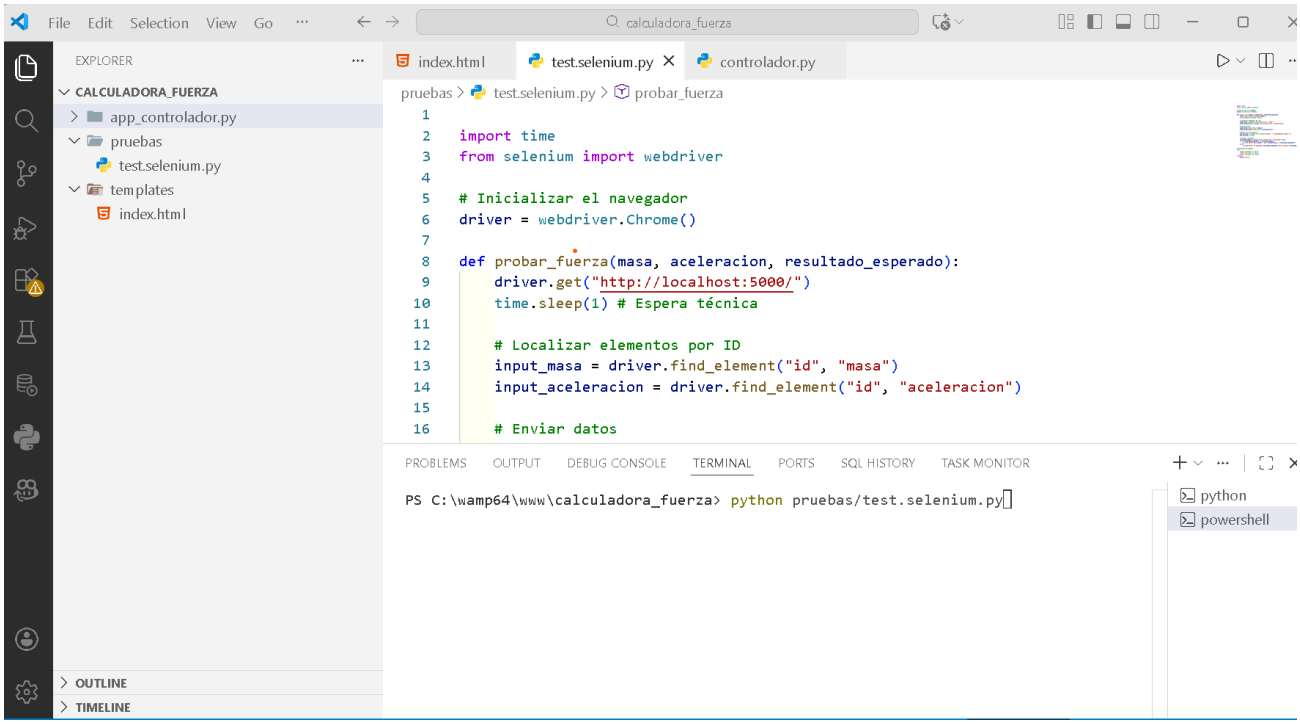
- Para buscar el puerto * Running on http://127.0.0.1:5000/ (Press CTRL+C to quit)

Aplicación de Primera prueba manual funcional,

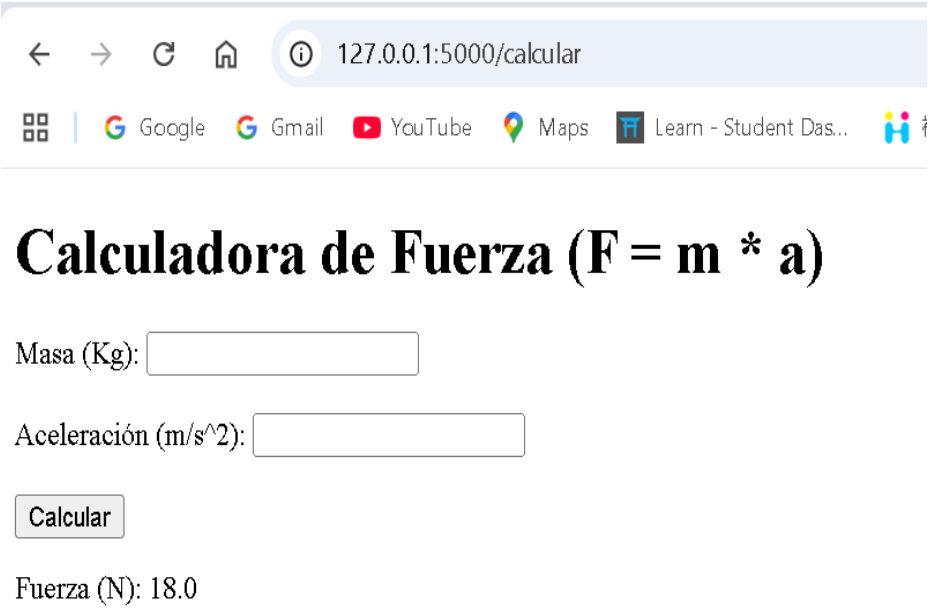




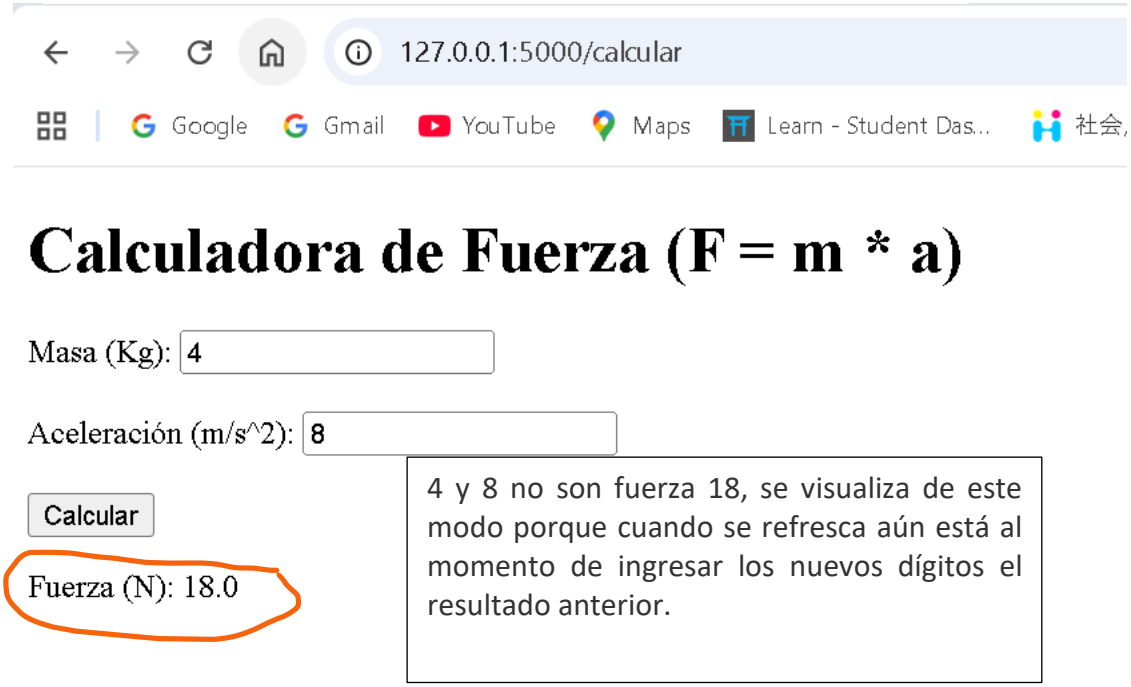
Se abrió un segundo terminal para que **Selenium WebDriver** abra una ventana de Chrome controlada automáticamente.



- Se diseñan casos de prueba que evalúen el resultado obtenido frente al resultado esperado, determinando así si la prueba fue exitosa o fallida y que cubran una amplia gama de escenarios posibles:



- Al apretar botón Calcular los dígitos de entrada desaparecen.



4 y 8 no son fuerza 18, se visualiza de este modo porque cuando se refresca aún está al momento de ingresar los nuevos dígitos el resultado anterior.

Al demostrar si la funcionalidad de cálculo de fuerza produce resultados precisos y coherentes, y el cálculo es correcto ingresando valores int, solo positivos se ve un resultado exitoso.

```
19        <p id="resultado">Fuerza (N): {{ fuerza }}</p>
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SQL HISTORY TASK MONITOR

```
ÉXITO: Masa 8 * Acel 10 = 80.0
PS C:\wamp64\www\calculadora_fuerza> python pruebas/test.selenium.py
ÉXITO: Masa 2 * Acel 9 = 18.0
ÉXITO: Masa 4 * Acel 8 = 32.0
ÉXITO: Masa 8 * Acel 10 = 80.0
PS C:\wamp64\www\calculadora_fuerza>  python pruebas/test.selenium.py
ÉXITO: Masa 2 * Acel 9 = 18.0
ÉXITO: Masa 4 * Acel 8 = 32.0
ÉXITO: Masa 8 * Acel 10 = 80.0
PS C:\wamp64\www\calculadora_fuerza> ^C
```

python
powershell
Python Debug ...

- Gestión de datos de prueba, agregando tantas pruebas y datos como sean necesarios para validar la robustez y la precisión de la aplicación.

127.0.0.1:5000/calcular

Google Gmail YouTube Maps Learn - Student Das... 社会,媒体,商标githu...

Calculadora de Fuerza (F = m * a)

Masa (Kg): 8

Aceleración (m/s^2): 10

Calcular

Fuerza (N): 32.0

El mismo caso anterior.

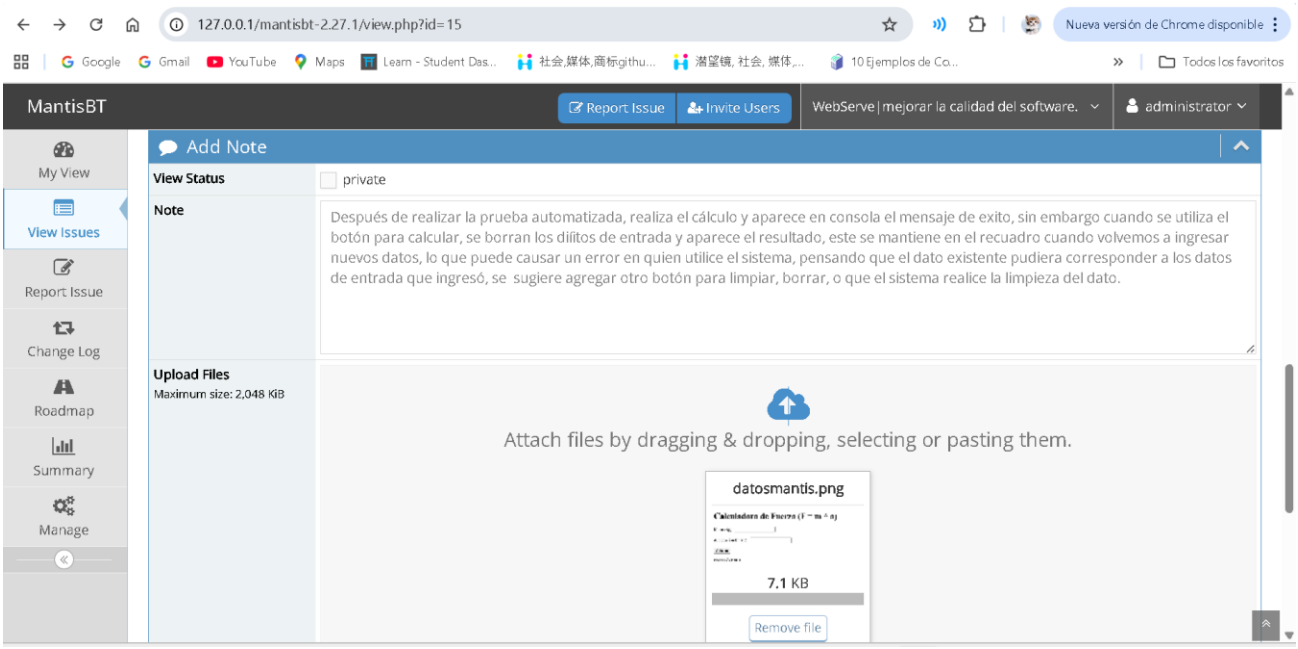
Calculadora de Fuerza (F = m * a)

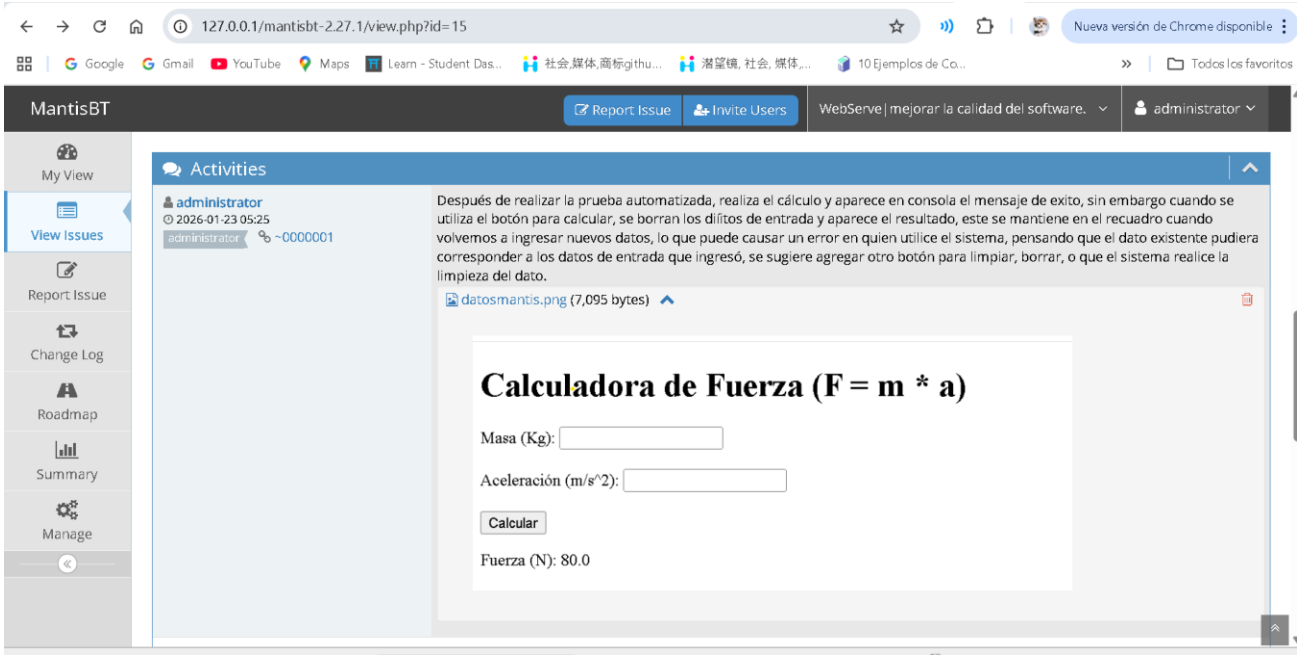
```
PS C:\wamp64\www\calculadora_fuerza> python pruebas/test.selenium.py
ÉXITO: Masa 2 * Acel 9 = 18.0
ÉXITO: Masa 4 * Acel 8 = 32.0
ÉXITO: Masa 8 * Acel 10 = 80.0
PS C:\wamp64\www\calculadora_fuerza>
```

Fuerza (N): 80.0

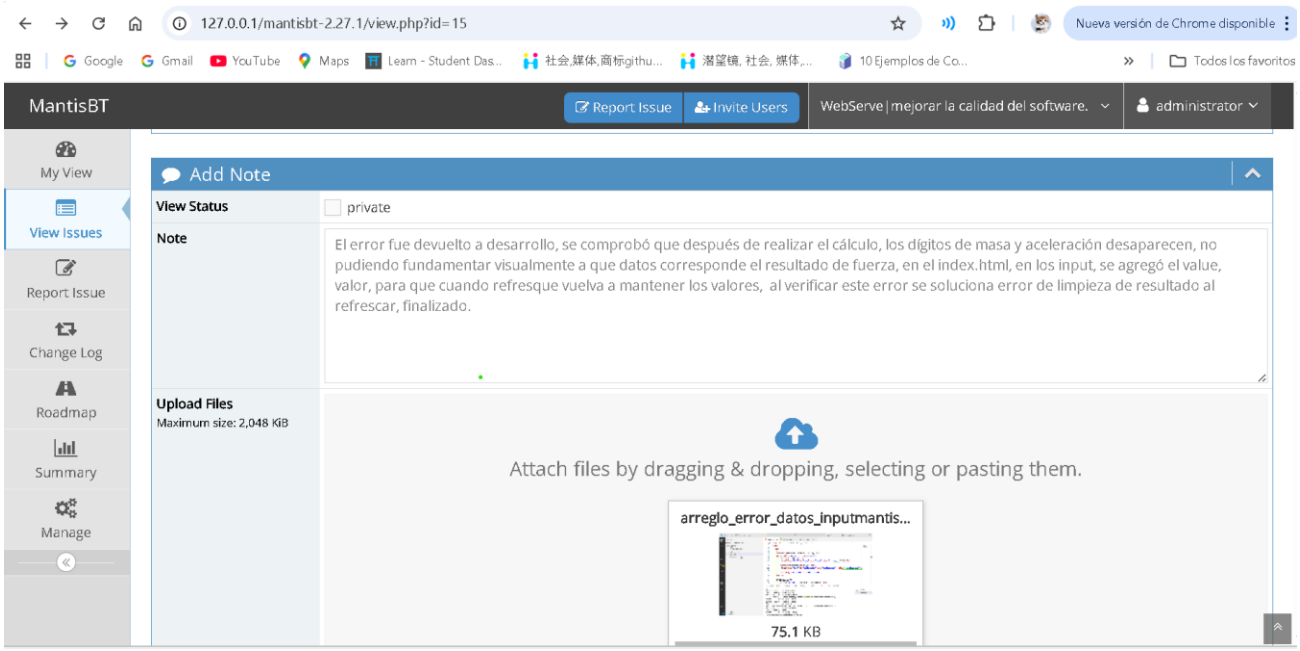
En view issues de Mantis correspondería hacer el reporte de la prueba automatizada, para la actualización, y continuar con el flujo de gestión de calidad.

Pruebas que cubran una amplia gama de escenarios posibles.

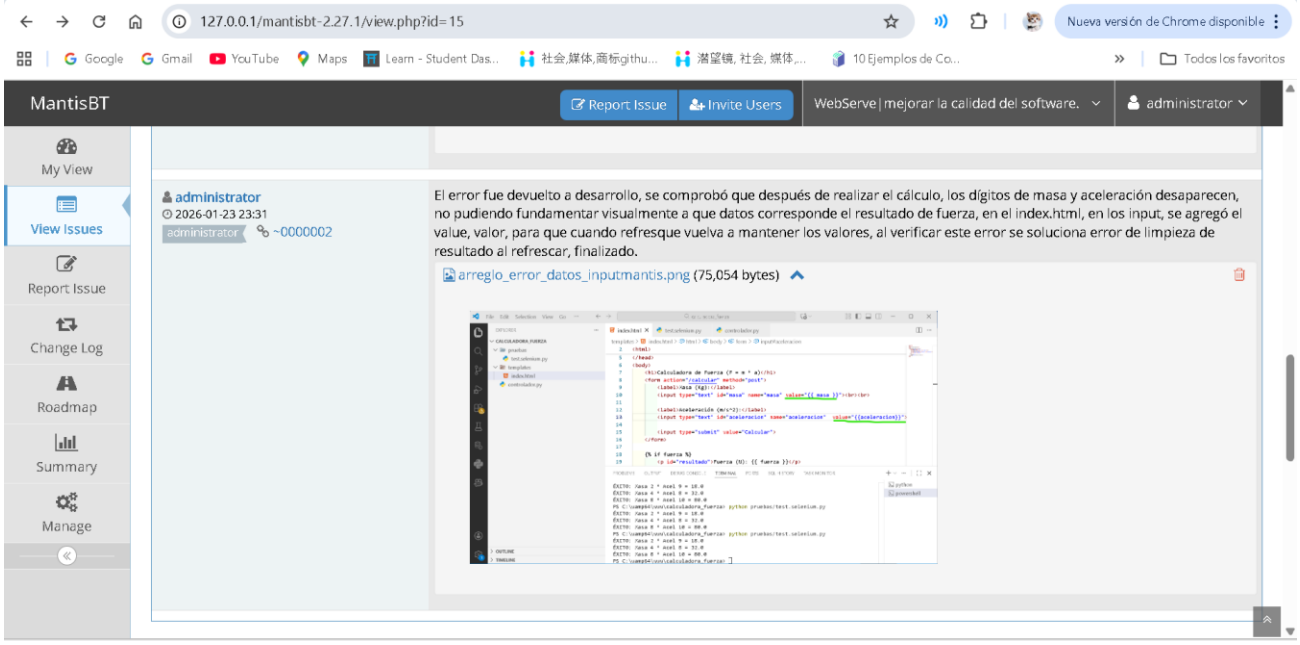


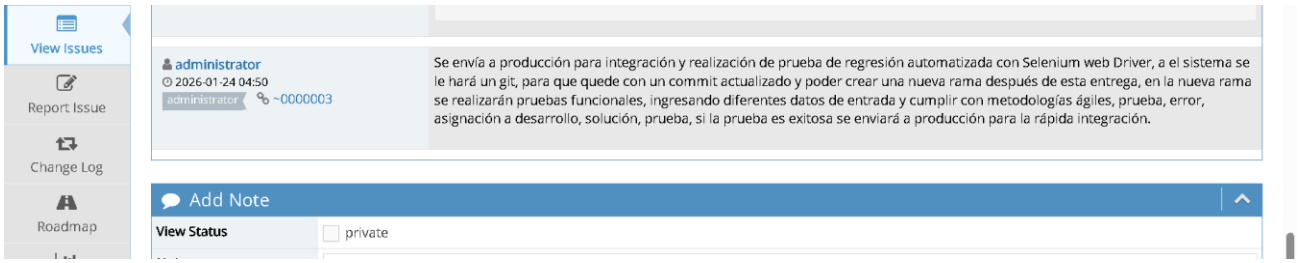


Error documentado y subido.

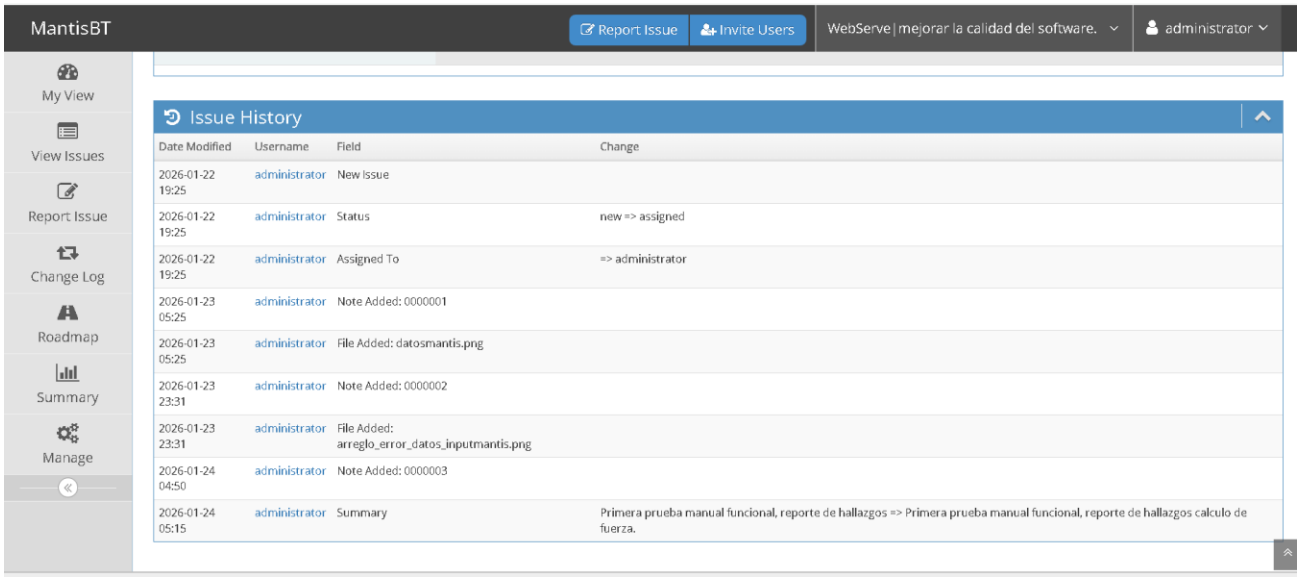


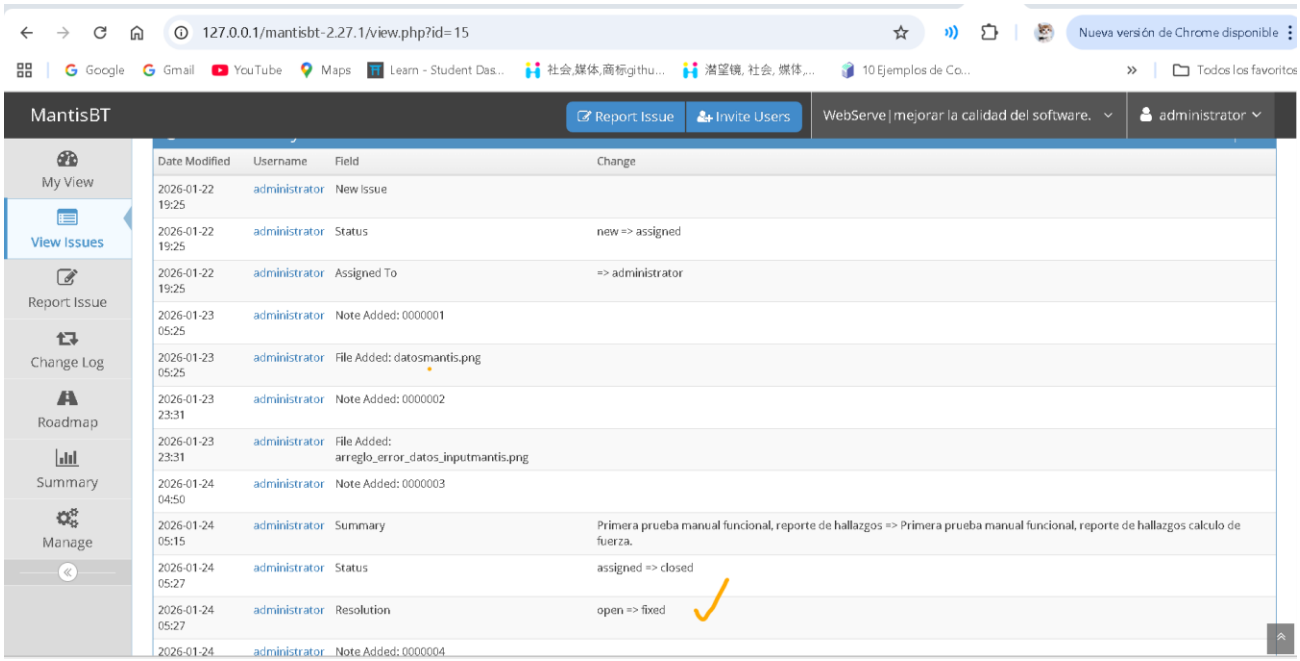
Adjuntado a ciclo de vida del error,





Se cierra el ciclo de vida de ese error como resolved, y comparto captura de pantalla de su historial.





Al ingresar un string en vez de un int, el sistema produce un error.

ValueError

```
ValueError: could not convert string to float: 'a'
```

Traceback (most recent call last)

File "C:\Users\Amparo\AppData\Local\Programs\Python\Python310\lib\site-packages\flask\app.py", line *1536*, in `__call__`

```
return self.wsgi_app(environ, start_response)
```

File "C:\Users\Amparo\AppData\Local\Programs\Python\Python310\lib\site-packages\flask\app.py", line *1514*, in `wsgi_app`

```
response = self.handle_exception(e)
```

File "C:\Users\Amparo\AppData\Local\Programs\Python\Python310\lib\site-packages\flask\app.py", line *1511*, in `wsgi_app`

```
response = self.full_dispatch_request()
```

File "C:\Users\Amparo\AppData\Local\Programs\Python\Python310\lib\site-packages\flask\app.py", line *919*, in `full_dispatch_request`

```
rv = self.handle_user_exception(e)
```

File "C:\Users\Amparo\AppData\Local\Programs\Python\Python310\lib\site-packages\flask\app.py", line *917*, in `full_dispatch_request`

```
rv = self.dispatch_request()
```

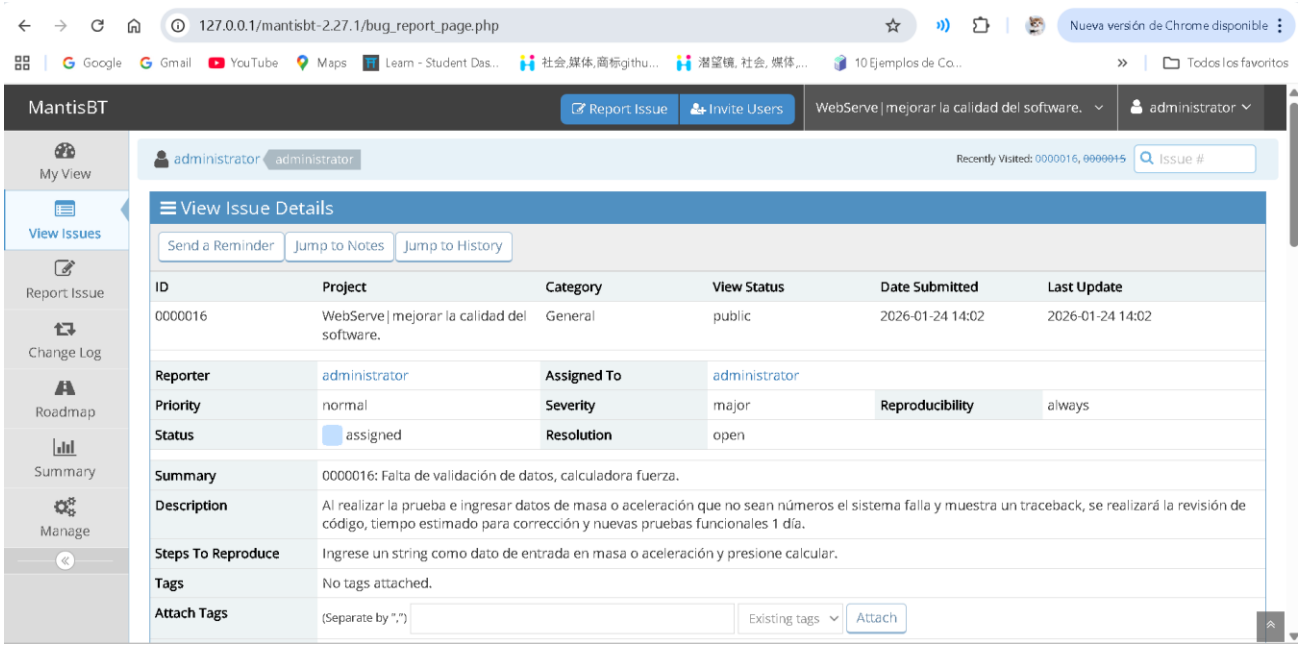
File "C:\Users\Amparo\AppData\Local\Programs\Python\Python310\lib\site-packages\flask\app.py", line *902*, in `dispatch_request`

```
return self.ensure_sync(self.view_functions[rule.endpoint])(**view_args)  # type: ignore[no-any-return]
```

File "c:\wamp64\www\calculadora_fuerza\controlador.py", line *12*, in `calcular`

```
masa = float(request.form['masa'])
```

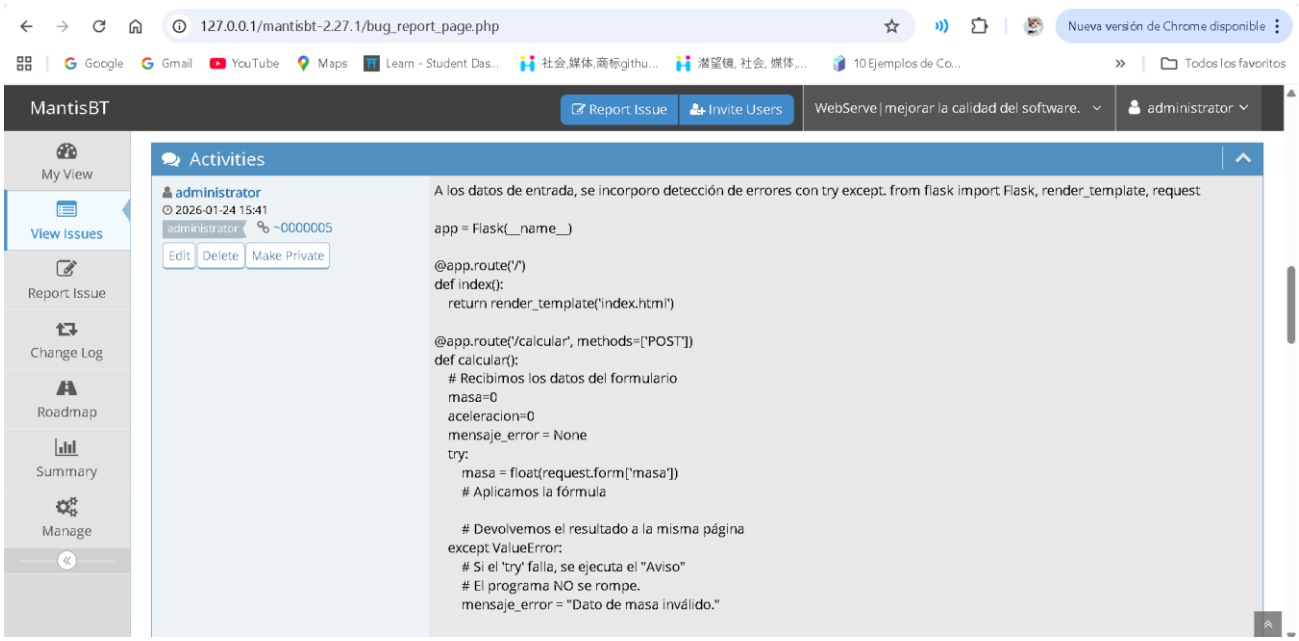
Nuevo issue,

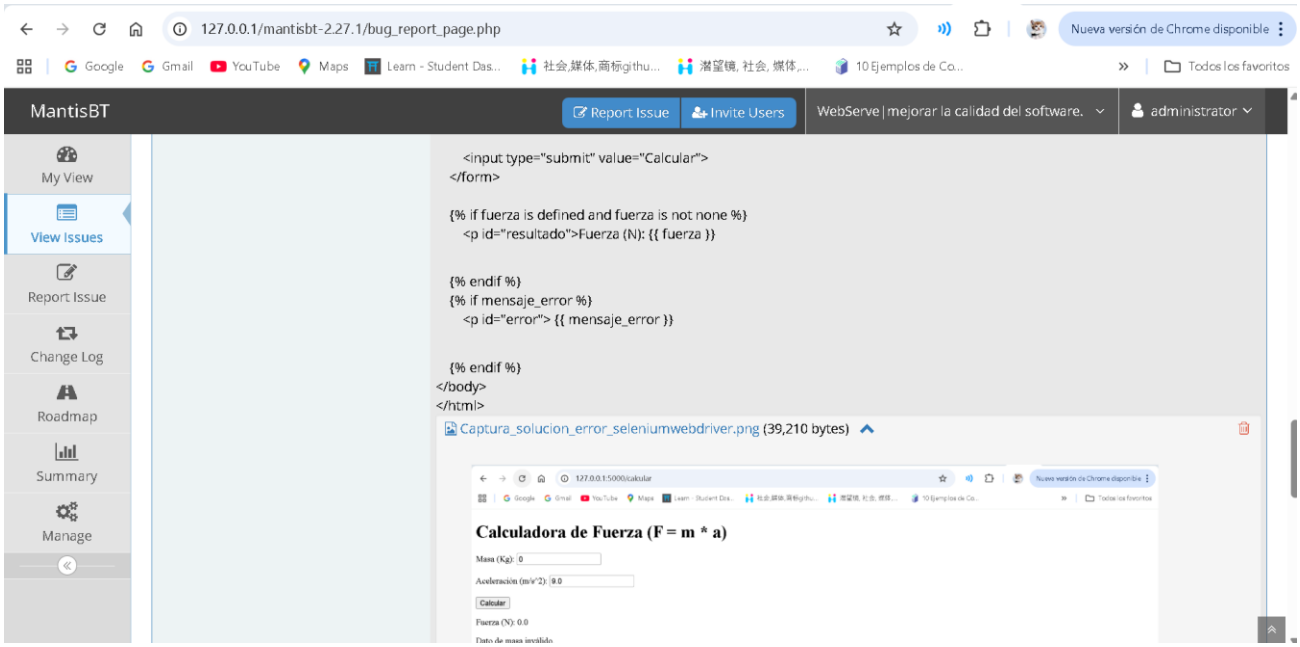


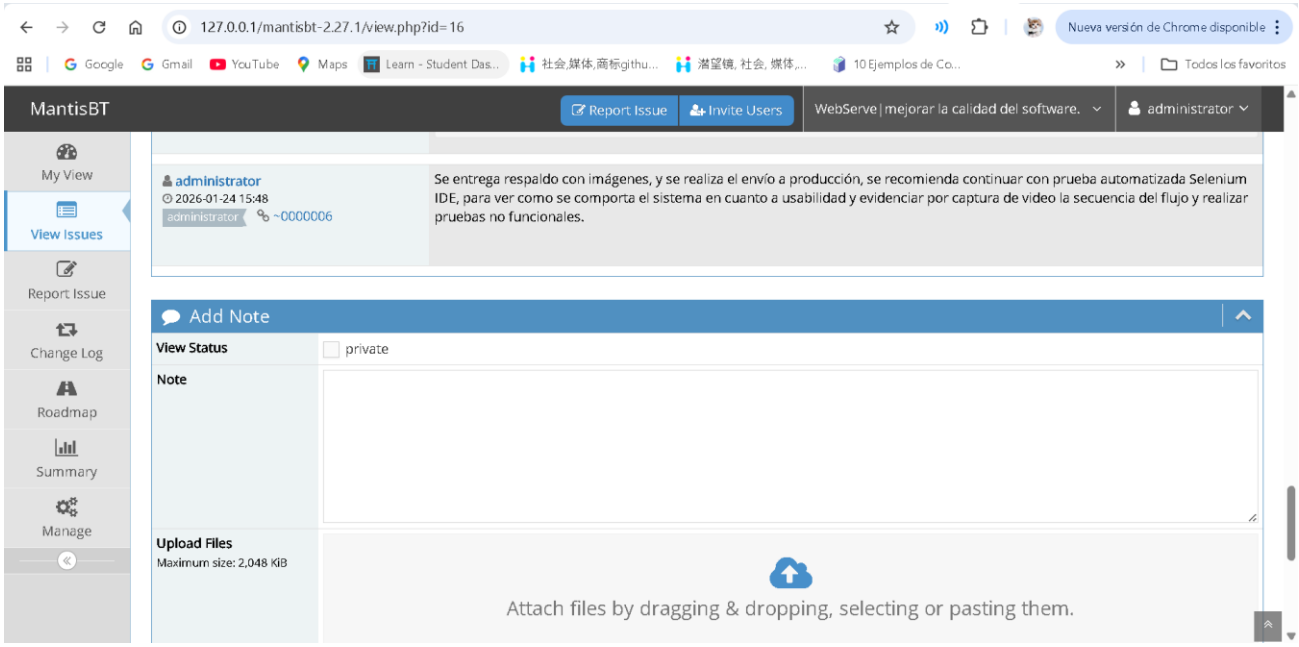
Reporte nuevo error,

- La aplicación debe manejar correctamente diferentes combinaciones de valores de masa y aceleración, pero no genera el resultado esperado de acuerdo con la fórmula establecida.

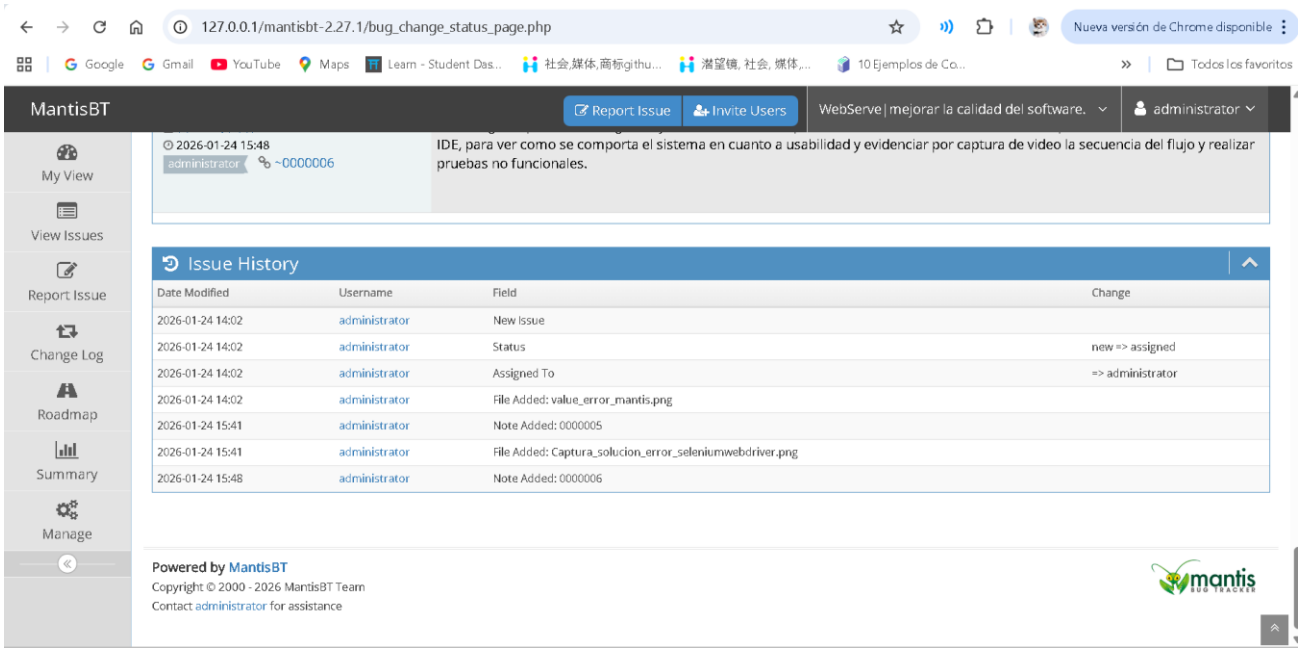
Reporte solución nuevo error, validación datos de entrada.



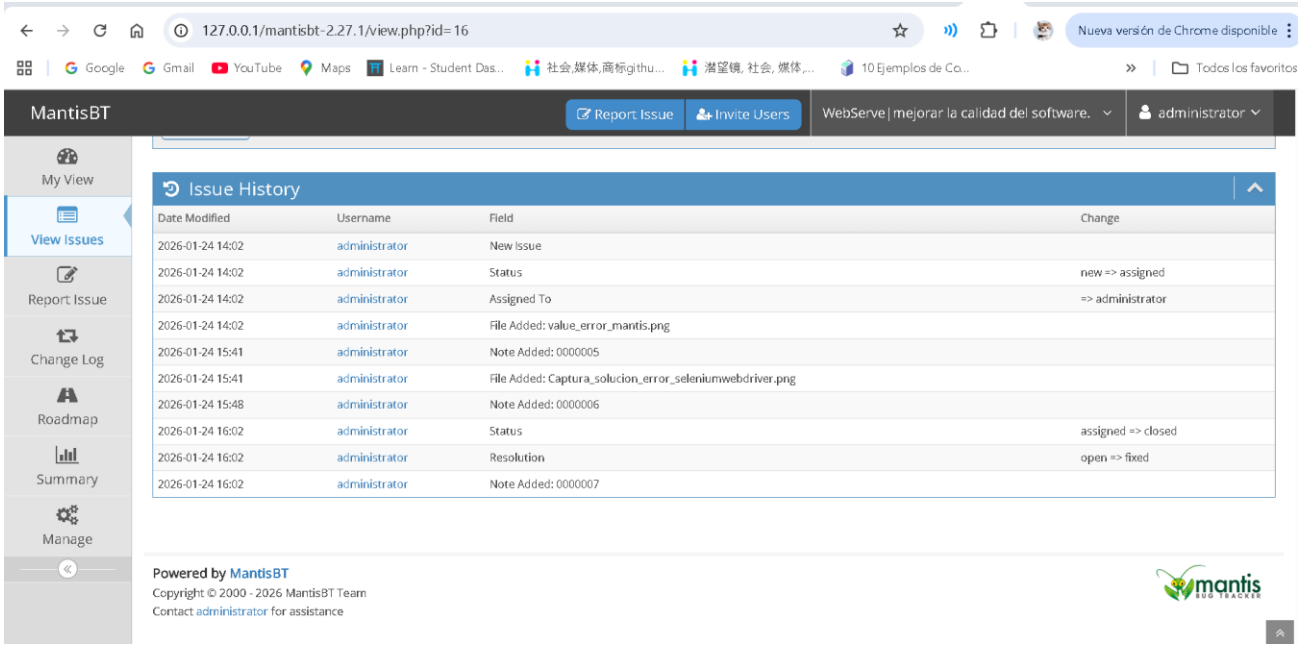



127.0.0.1/mantisbt-2.27.1/view.php?id=16
MantisBT
Report Issue
Invite Users
WebServe | mejorar la calidad del software.
administrator
My View
View Issues
Report Issue
Change Log
Roadmap
Summary
Manage
administrator
2026-01-24 15:48
~0000006
Se entrega respaldo con imágenes, y se realiza el envío a producción, se recomienda continuar con prueba automatizada Selenium IDE, para ver como se comporta el sistema en cuanto a usabilidad y evidenciar por captura de video la secuencia del flujo y realizar pruebas no funcionales.
Add Note
View Status
private
Note
Upload Files
Maximum size: 2,048 KiB
Attach files by dragging & dropping, selecting or pasting them.

Historial,


127.0.0.1/mantisbt-2.27.1/bug_change_status_page.php
MantisBT
Report Issue
Invite Users
WebServe | mejorar la calidad del software.
administrator
2026-01-24 15:48
~0000006
IDE, para ver como se comporta el sistema en cuanto a usabilidad y evidenciar por captura de video la secuencia del flujo y realizar pruebas no funcionales.
Issue History
Date Modified | Username | Field | Change
2026-01-24 14:02 | administrator | New Issue
2026-01-24 14:02 | administrator | Status | new => assigned
2026-01-24 14:02 | administrator | Assigned To | => administrator
2026-01-24 14:02 | administrator | File Added: value_error_mantis.png
2026-01-24 15:41 | administrator | Note Added: 0000005
2026-01-24 15:41 | administrator | File Added: Captura_solucion_error_seleniumwebdriver.png
2026-01-24 15:48 | administrator | Note Added: 0000006
Powered by MantisBT
Copyright © 2000 - 2026 MantisBT Team
Contact administrator for assistance

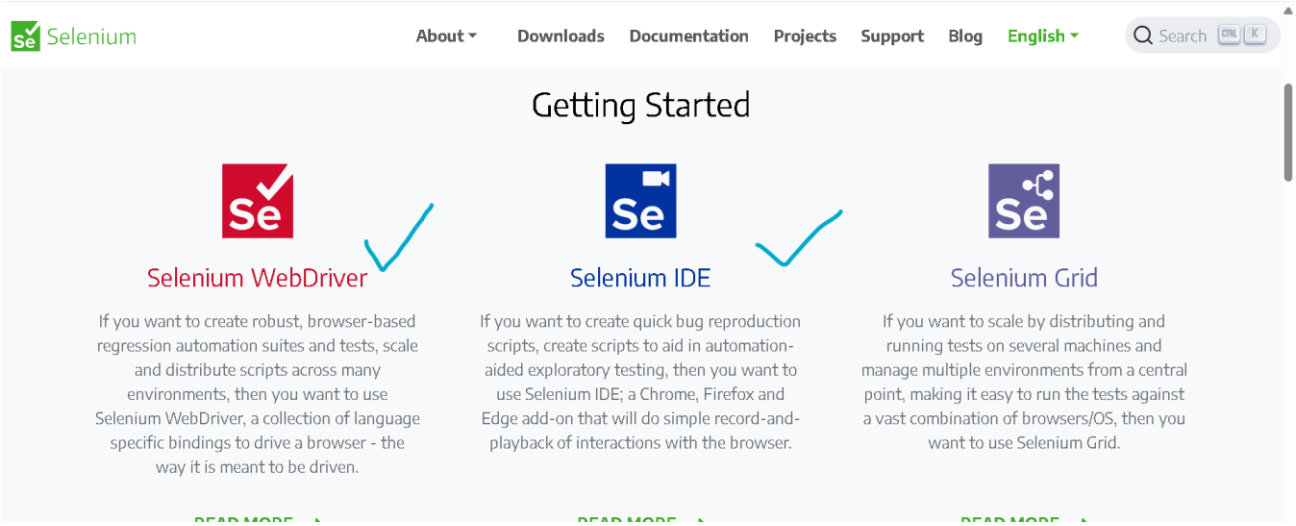
Issue Cerrado,



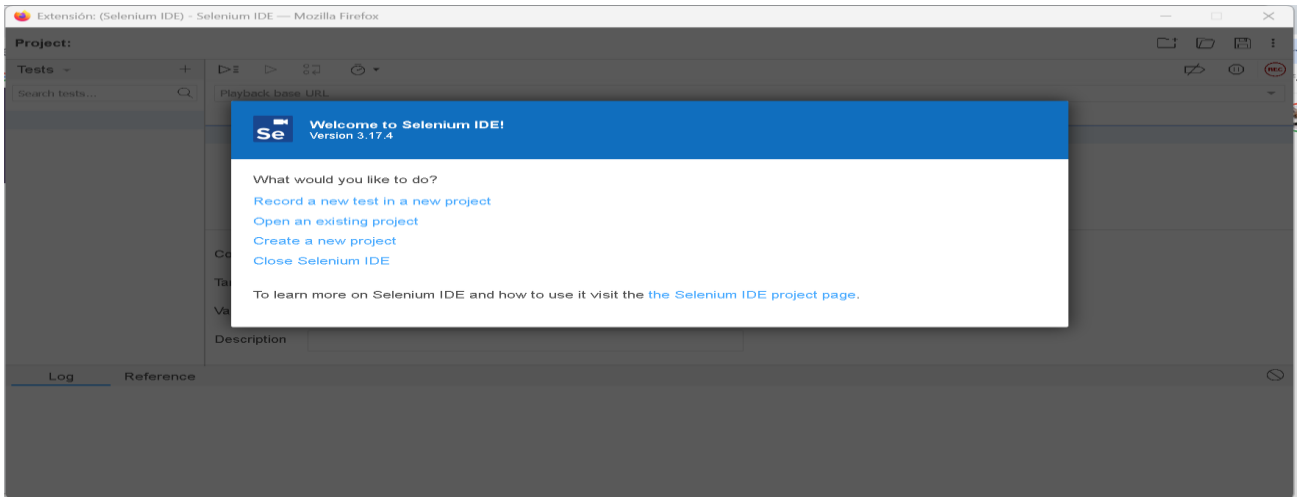
Tras haber gestionado el ciclo de vida de los errores en el punto anterior, se hizo evidente la necesidad de una herramienta de mayor agilidad visual y potencial de registro tanto de línea de ejecución como la evidencia en video, la resolución del problema planteado requirió de una lógica del sistema de análisis, y es en esta parte del proceso que se detallan las características de Selenium IDE que permitieron esta validación final y dirigió el orden lógico para las preguntas 3 y 2.

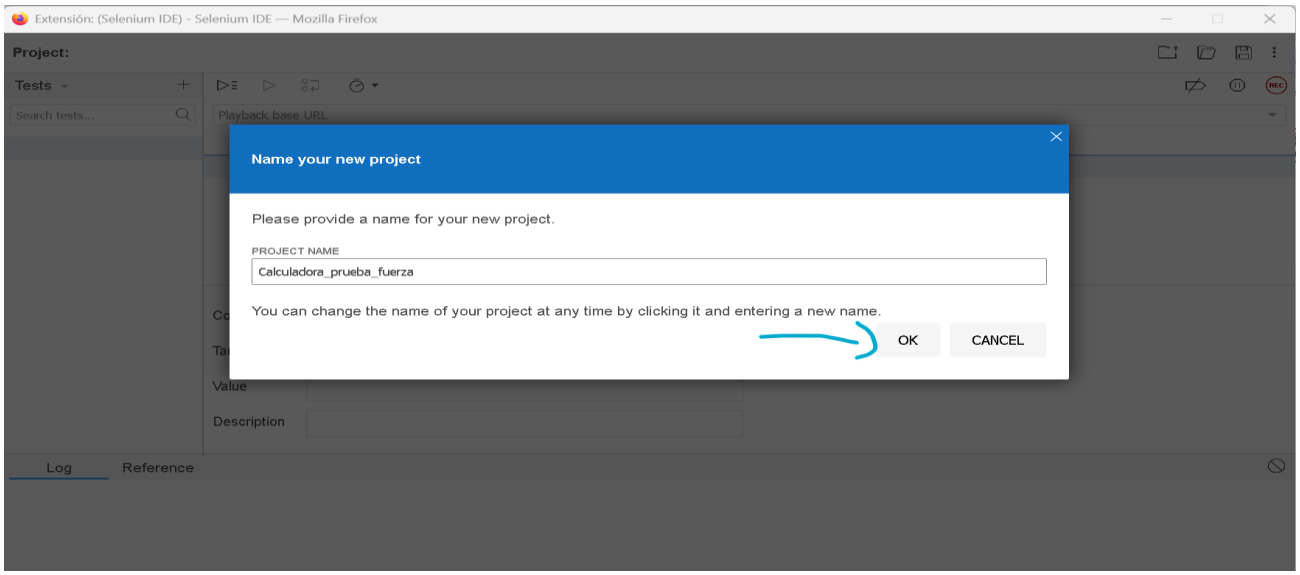
2. ¿Cuáles son las características clave de Selenium IDE que la convierten en una herramienta eficaz para la detección de errores del software en el caso práctico de pruebas automatizadas?

Según la página oficial de Selenium podemos ver las distintas versiones de la aplicación de la cual se van a realizar pruebas en Selenium webdrivery Selenium IDE, Selenium Grid es para probar sistemas completos.



Selenium IDE,





Repetir pruebas grabadas, todo, actual.

Se pueden ver los pasos que se grabaron para cada situación .

Velocidad ejecución.

Graba

La acción que se realiza ej. open para abrir la web, type para escribir, click para el botón.

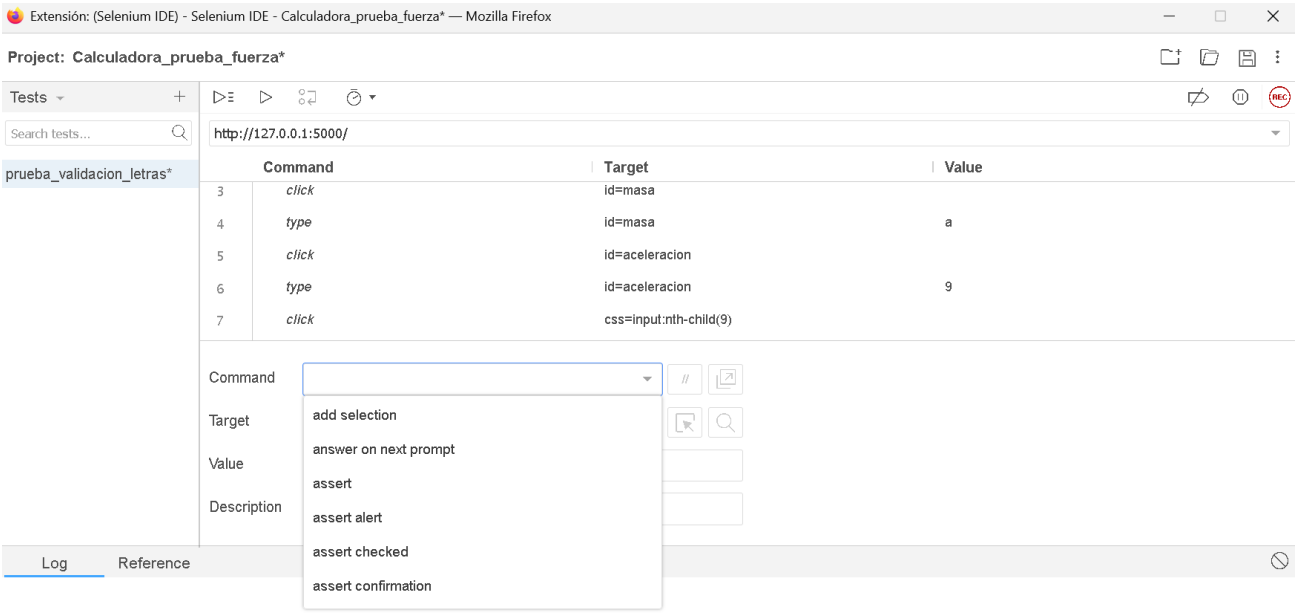
El IDE usa los id, como id=masa o id=aceleracion.

El dato que ingresa, por ejemplo, el número 10. o la letra A

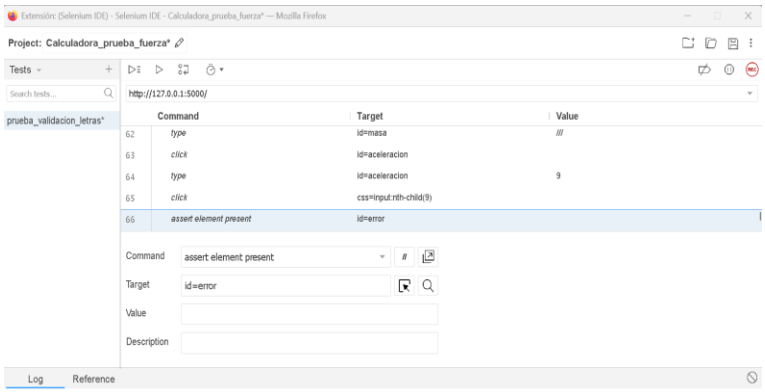
Dice en tiempo real si el paso funcionó sale en verde o si falló, sale en rojo.

Explica exactamente para qué sirve ese comando.

Proceso de primeras pruebas con diferentes datos de entrada, type dato click infiere acción.



El commando assert element present, busca el elemento presente con el target id=error, que programé en el try-except, de index.html, si el programa se hubiera roto, el elemento no aparecería en la lista y la prueba habría fallado.

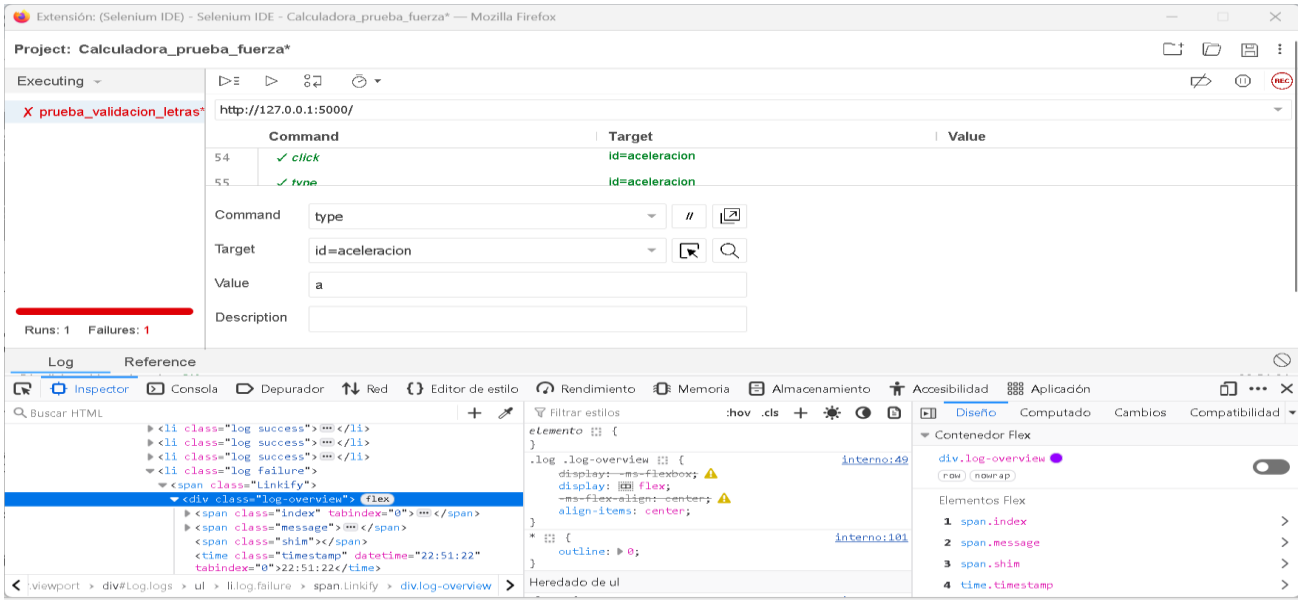


"Run current test". Sirve para ejecutar únicamente la prueba abierta, en este caso, la prueba_validacion_letras.

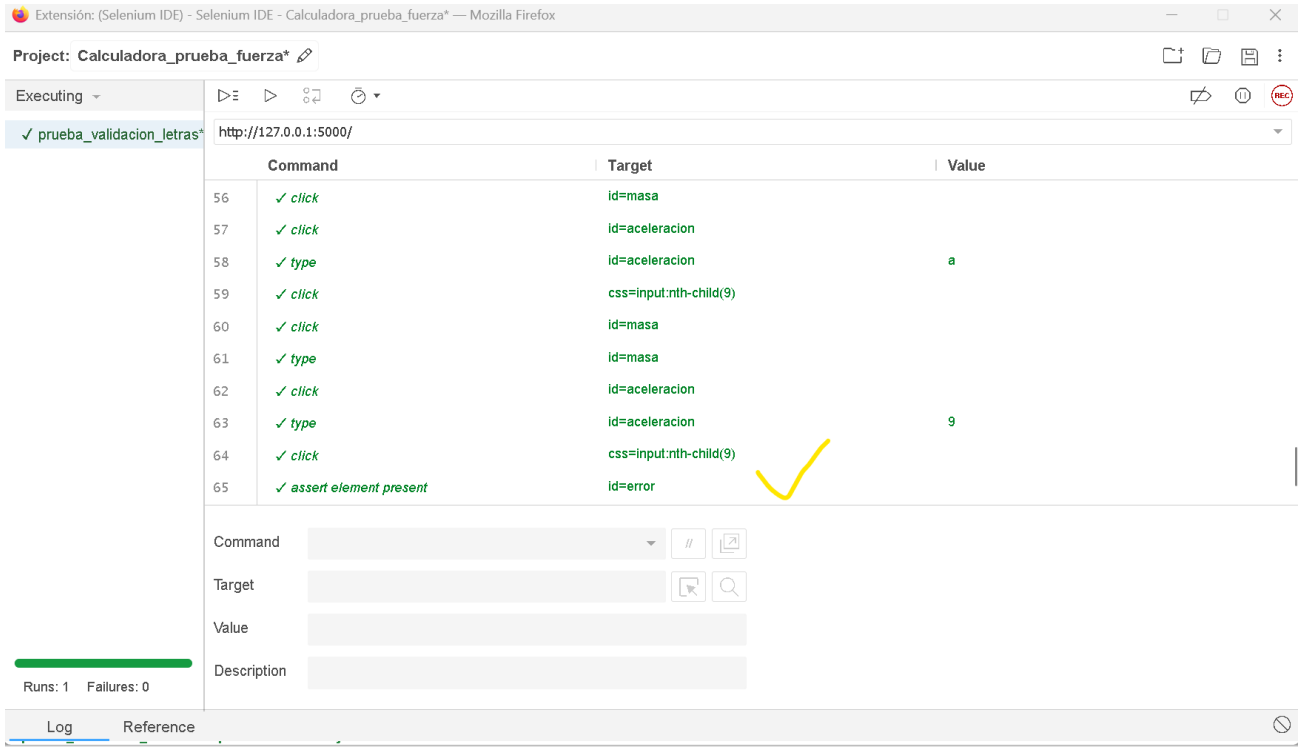
"Run all tests". En muchas pruebas grabadas las ejecuta todas una tras otra para ver si algo falló.

- El soporte para aserciones, se demuestra al usar assert element present para validar el error.
- La Interfaz de usuario es amigable, ya que algunas acciones se pueden inferir, es apropiado para la curiosidad natural ya que si posiciono el mouse encima de los iconos tiene una frase descriptiva y sobre todo los elementos no están tan juntos lo que permite al usuario de Selenium IDE no sentirse abrumado de información, se configuró la validación sin escribir código innecesario, solo se seleccionó el comando, mediante Selenium IDE, se confirma que ante entradas inválidas como letras, el bloque try-except del controlador activa correctamente el mensaje de error en la interfaz id=error que se estableció en jinja, como anteriormente se señaló index.html evitando la caída del servidor y cumpliendo con los estándares de calidad que se espera.

Inspeccionando Selenium IDE.

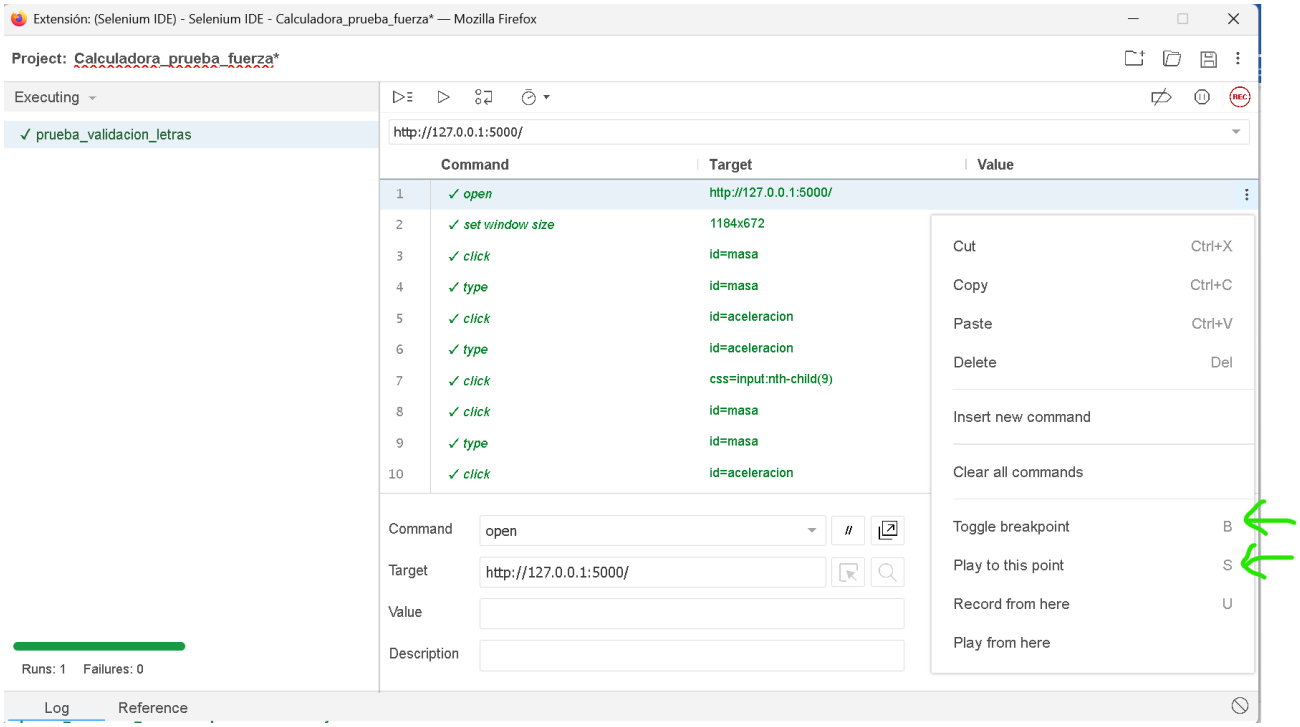


En la demostración de las funcionalidades de Selenium IDE, Al mostrar el assert element presente en la imagen funcionando en verde, se prueba el Soporte para aserciones,



La presente condición contextualizada y a mi entender el caso práctico que significaría llevar a cabo y la aplicación de los principios para el desarrollo del análisis de QA dan paso a la comprensión y el análisis de:

Selenium IDE permite agregar **Checkpoints,** que en el caso anterior tuve que realizar con Visual Studio de forma manual o verificaciones visuales (Assertions) para asegurar que, por ejemplo, el mensaje de error aparezca realmente en la pantalla y no solo en la consola.



- Prueba de interacción, prueba no funcional, con la interfaz de usuario utilizando Selenium para realizar pruebas automatizadas.

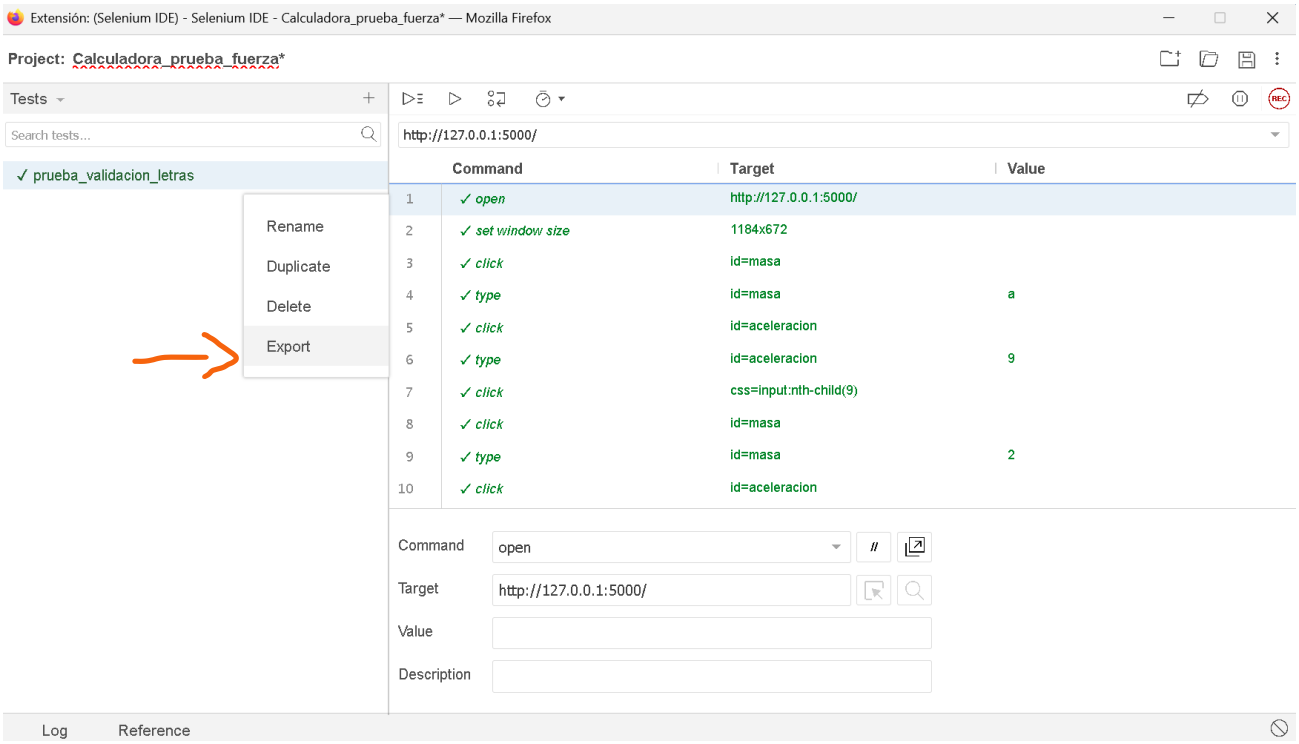
El objetivo es probar que el mensaje de error del try-except si funciona, entonces se debe usar assert text,

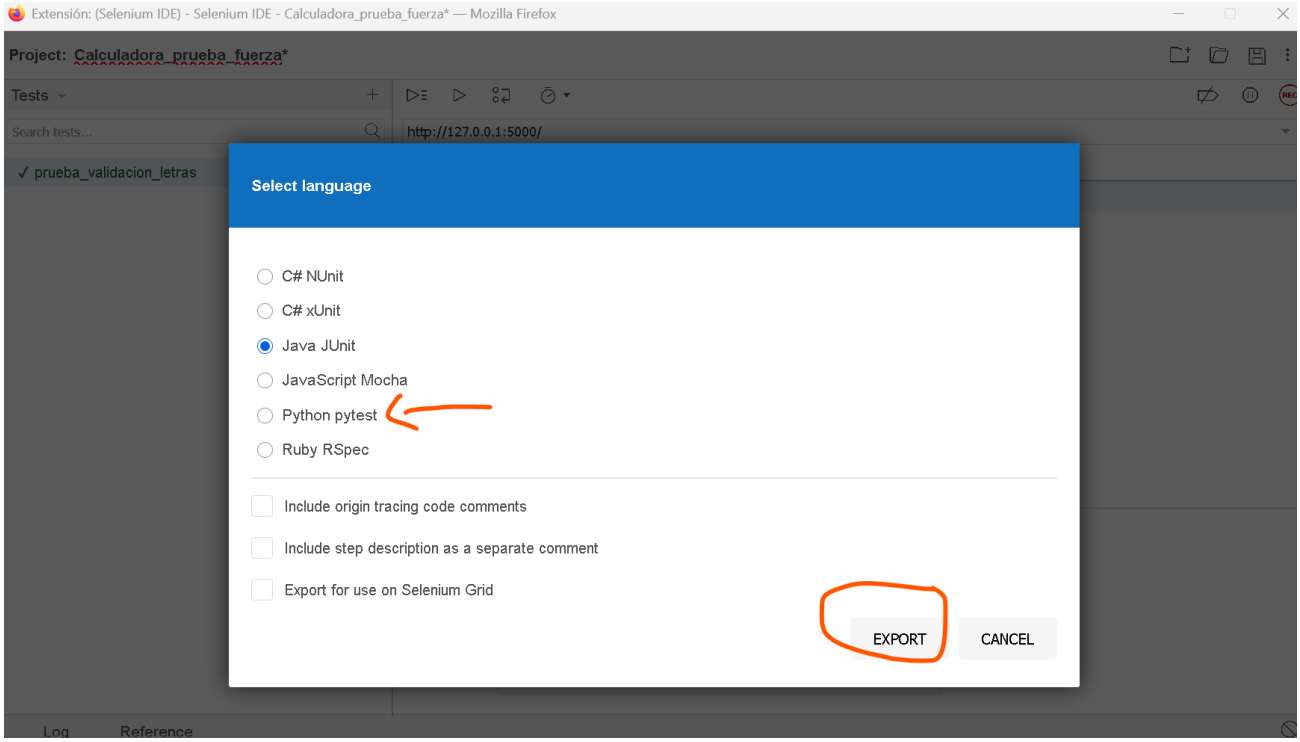
- Dentro del soporte con múltiples navegadores puedo constatar que en Google Crome se me dificultó la descarga, aún así, el IDE y el localhost:5000, lo tenía abierto en Crome y bajo ninguna circunstancia tuve algún problema al descargar o ejecutar la

extensión y correr Selenium IDE desde Firefox, donde se realizó la descarga y se **Prueba la Compatibilidad de Navegadores** el Cross-Browser Testing.

- Según el material de profundización del ramo en curso semana 2, las pruebas realizadas en Selenium IDE y la capacidad de exportación "Permite integrar las pruebas con frameworks de automatización más avanzados o utilizarlas en diferentes entornos de desarrollo", la grabación y reproducción, proceso que se usa para generar automáticamente la visualización y respaldo de las pruebas invalidantemente necesario para realizar correctas regresiones, cómo realizarlo **Exportar a Python,** clic derecho sobre la prueba y seleccionar **Export -> Python pytest**. el código generado es el que se adjunta en la descripción de Mantis y así documentar que la prueba ha sido automatizada y realizada, también es posible que el proyecto de Selenium IDE se guarde como un archivo con extensión .side, subirlo a Mantis como un archivo adjunto para ser compartido, y el equipo lo descargue y lo corra en su propio computador.
- Para integrar con frameworks más avanzados como Selenium Grid, grabando en Selenium IDE exportando a código Python.

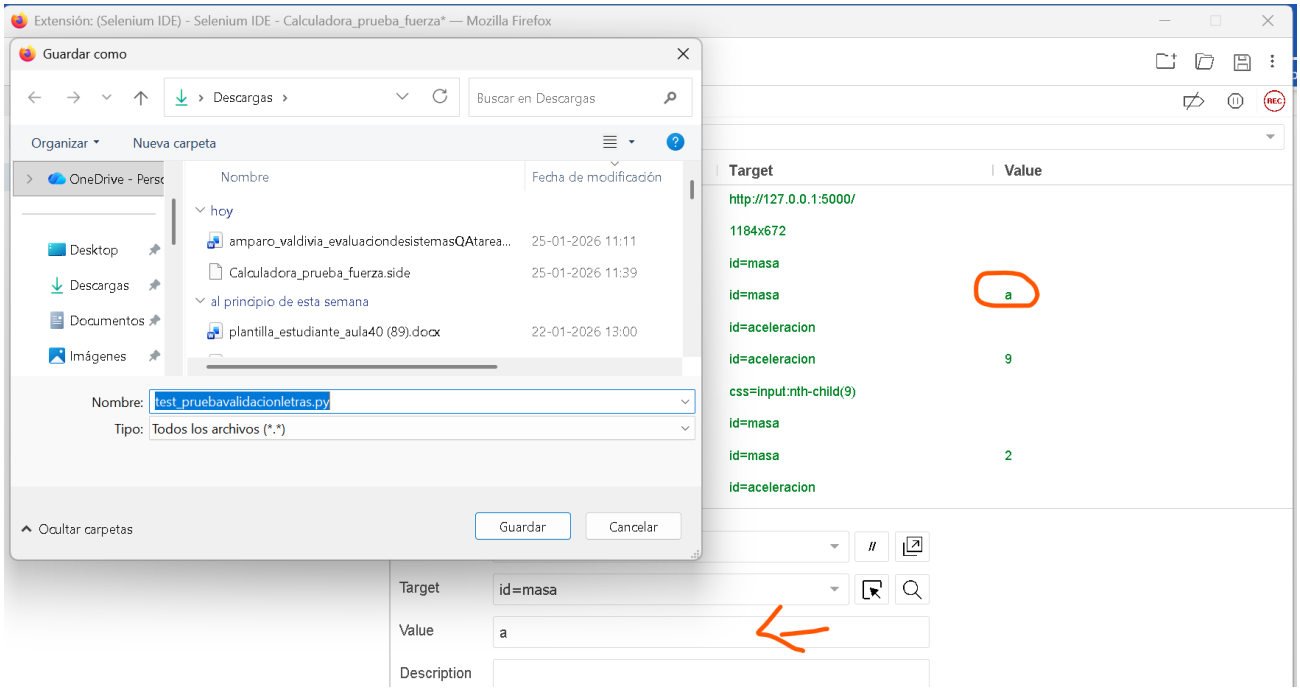
  Para encontrar algunos iconos y funciones en Selenium IDE también hay que aplicar ingeniería inversa y ser proactivo a la regresión aplicable a todo contexto, frase propia de superación al desafio.

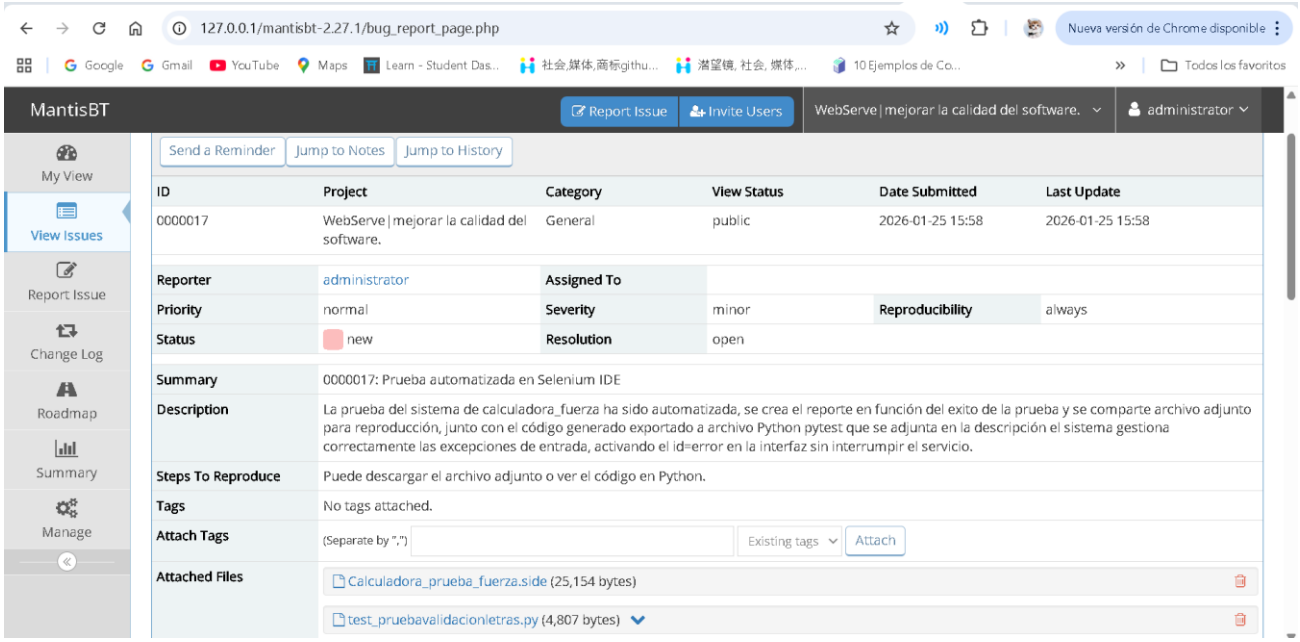

Extensión: (Selenium IDE) - Selenium IDE - Calculadora_prueba_fuerza* — Mozilla Firefox
Project: Calculadora_prueba_fuerza*
Tests
Search tests...
✓ prueba_validacion_letras
Rename
Duplicate
Delete
Export
http://127.0.0.1:5000/
Command
Target
Value
1 ✓ open http://127.0.0.1:5000/
2 ✓ set window size 1184x672
3 ✓ click id=masa
4 ✓ type id=masa a
5 ✓ click id=aceleracion
6 ✓ type id=aceleracion 9
7 ✓ click css=input:nth-child(9)
8 ✓ click id=masa
9 ✓ type id=masa 2
10 ✓ click id=aceleracion
Command open
Target http://127.0.0.1:5000/
Value
Description
Log
Reference


Extensión: (Selenium IDE) - Selenium IDE - Calculadora_prueba_fuerza* — Mozilla Firefox
Project: Calculadora_prueba_fuerza*
Tests
Search tests...
✓ prueba_validacion_letras
http://127.0.0.1:5000/
Select language
C# NUnit
C# xUnit
Java JUnit
JavaScript Mocha
Python pytest
Ruby RSpec
Include origin tracing code comments
Include step description as a separate comment
Export for use on Selenium Grid
EXPORT
CANCEL
Log
Reference

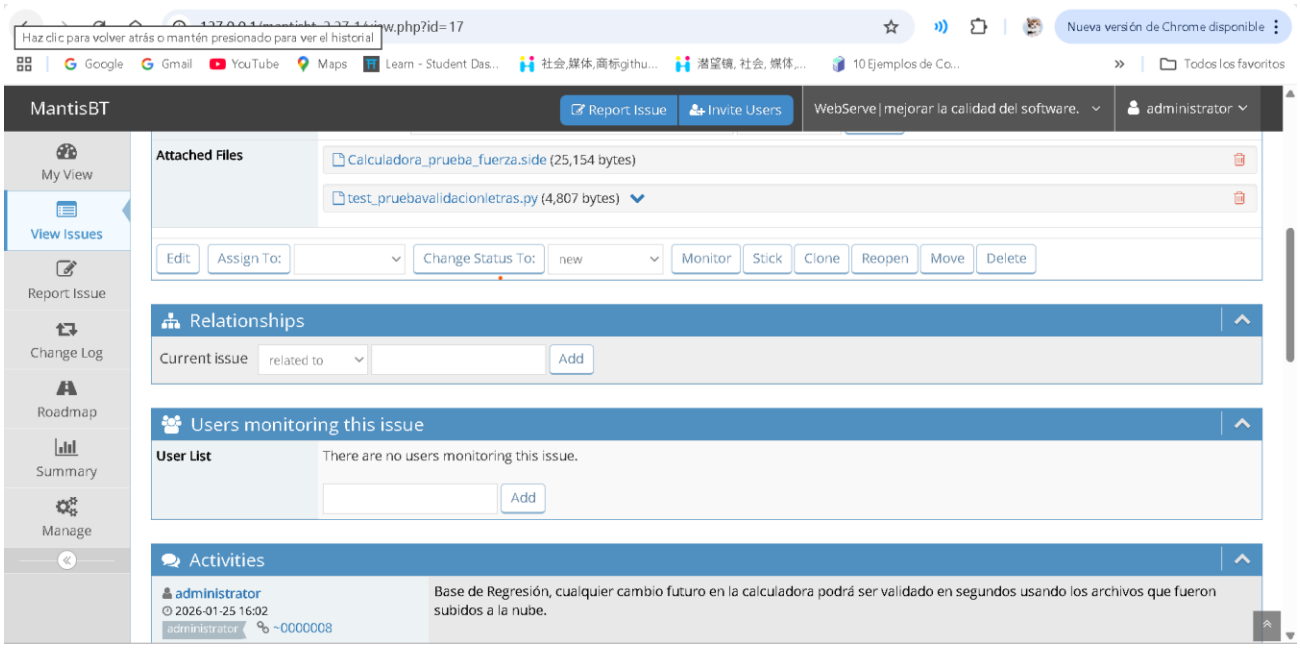
Esta es una muestra de la eficiencia de Selenium IDE que no permite exportar contenido con ítems vacíos, validación de integridad.

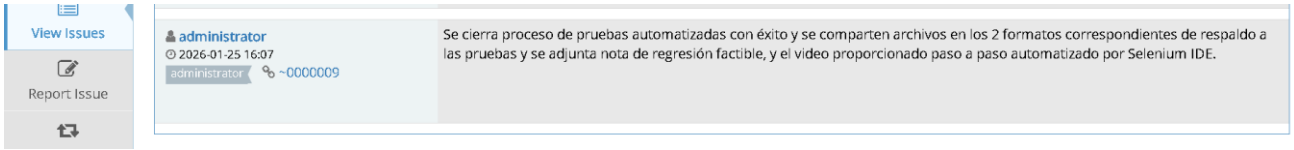
Para realizar la corrección volver al hábitat y hacer click en la línea y rellenar el value,

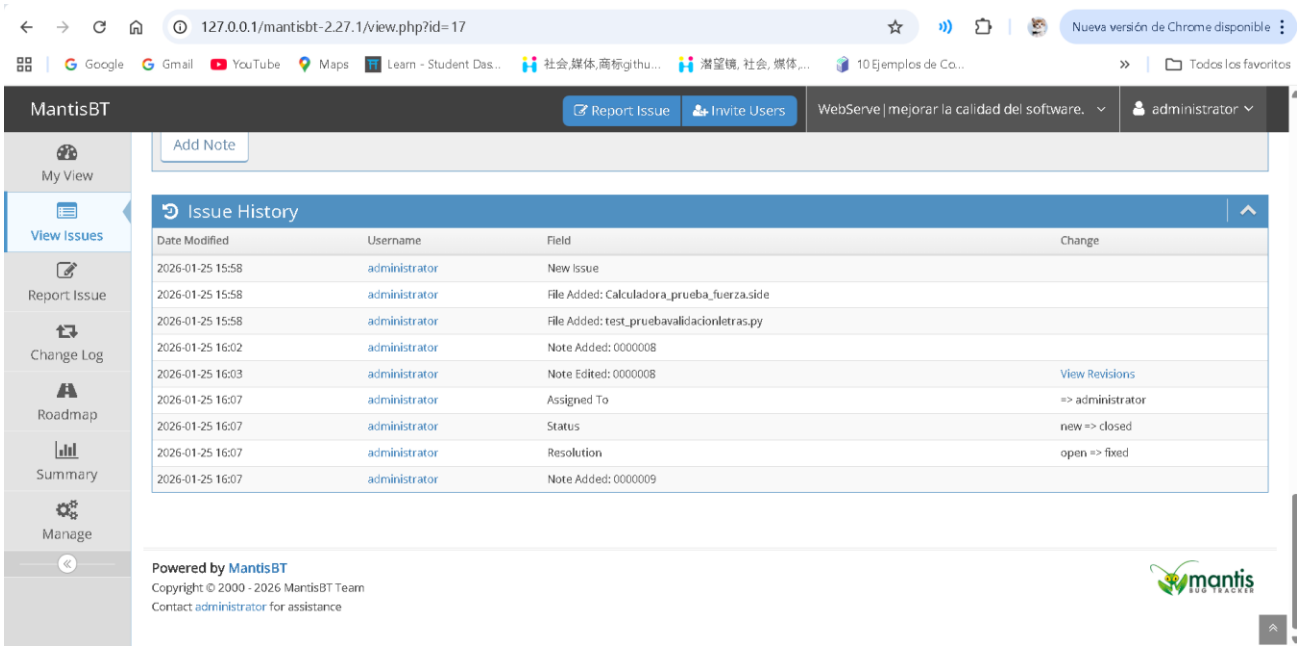


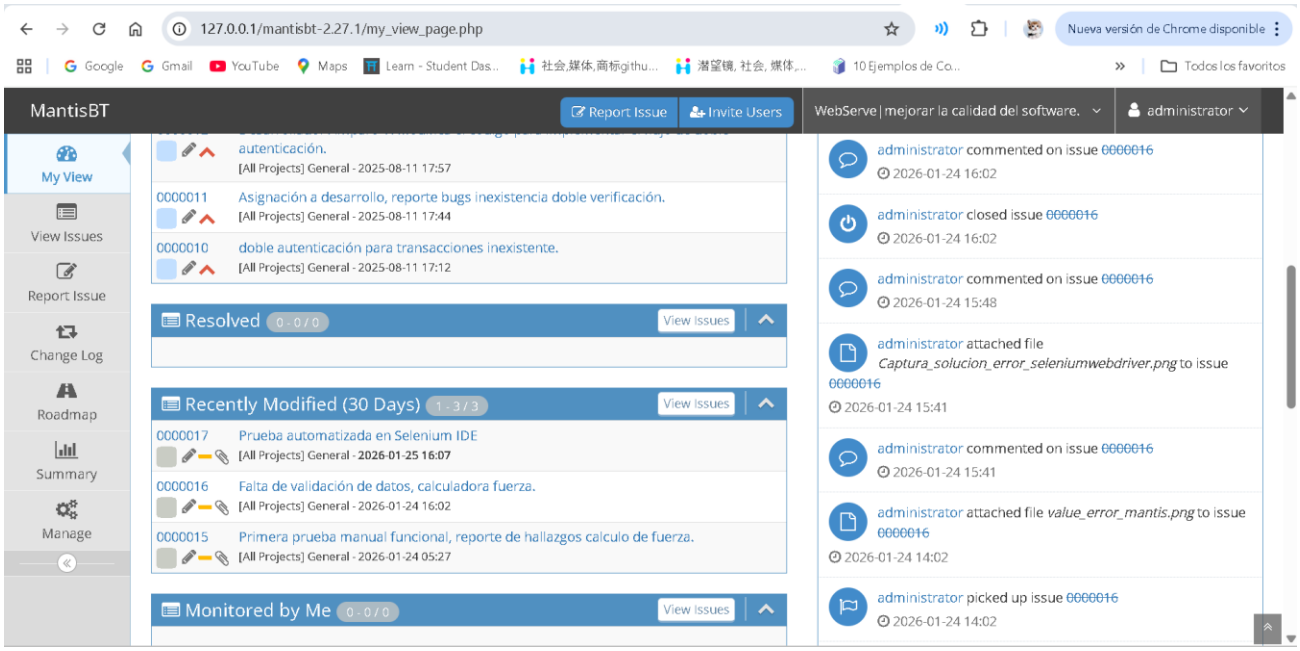


Se cierra el proceso de pruebas automatizadas y se adjunta imagen de Mantis con el ciclo de vida del error, el proceso de desarrollo de las pruebas, los hallazgos y la documentación pertinente a la realización solicitada, revisión de caso de automatización de Pruebas para una Aplicación web, perteneciente a una empresa de desarrollo de software, que realiza el cálculo de Fuerza, aplicación de herramientas y conceptos aprendidos, revisión de código fuente, solución de errores y mejoras pertinentes al caso.









https://www.loom.com/share/017260e7bf8f4270815305c837ebec44

REFERENCIAS BIBLIOGRÁFICAS

Selenium. (2026). Selenium: Automation for Browsers. https://www.selenium.dev/

Deming, W. E. (1989). Calidad, productividad y competitividad: La salida de la crisis. Editorial Díaz de Santos.

[IACC]. (2026). Automatización de pruebas y frameworks: Semana 2 [Material de estudio].

https://rise.articulate.com/share/VFO1EFR38HR2yGE7xz8YAgZ6iNtsIG5Y#/lessons/WSyH7OhN0nlALzeqEPtUi3IsT5xfcgY-

MantisBT Group. (2026). Mantis Bug Tracker (Versión 2.x) [Software de seguimiento de errores]. https://www.mantisbt.org/